

ResonantOS SDK Reference Demo

Technical Report and User Guide
SDK-DEMO-003

ResonantOS SDK Working Group

September 25, 2026

Abstract

This report documents SDK-DEMO-003, the reference SDK demonstration built against the current ResonantOS 2.0.0-alpha `dev` line. Three independently packaged local-service add-ons — Resonant Echo, Resonant Counter, and the SDK Guide — are discovered, validated, registered, granted, enforced, and revoked through the platform’s *existing* manifest, registry, capability, endpoint-guard, bridge, and extension mechanisms, with zero add-on-specific Core code and no parallel trust model. The report describes the objectives, the host-aligned architecture that was verified, the capability and isolation model, the engineering defects found and corrected during independent review, the adversarial red-team results, the test results, a step-by-step user guide, the honest demo-versus-production boundary, and a concrete way forward toward a production running SDK platform.

Contents

1	Objectives	2
2	Architecture Overview	3
2.1	Authority	3
2.2	Isolation and token delivery	4
3	The Reference Add-ons	4
3.1	Resonant Echo	4
3.2	Resonant Counter	4
3.3	SDK Guide	5
4	Capability and Security Model	5
5	How It Was Built	5
6	What Was Found	6
6.1	Resolved during the build	6
6.2	Verification-quality corrections	7
6.3	Deferred items (recorded, not lost)	7
7	Adversarial Red-Team	7
8	Test Results	8

9	User Guide	9
9.1	Prerequisites	9
9.2	Checkout and install	9
9.3	Start the three upstreams	9
9.4	Start the bridge	9
9.5	Load the extension	10
9.6	Walk the lifecycle	10
9.7	Run the test suite	10
10	Demo-vs-Production Honesty	10
11	The Way Forward Toward a Running SDK Platform	11
11.1	Close the enforcement gap (D1 + D3)	11
11.2	Token hygiene (D2)	11
11.3	Registry idempotency (D6)	11
11.4	CI wiring (D4)	11
11.5	Flake hygiene (D5)	11
11.6	Merge path	11
11.7	Publish the SDK contract	11
11.8	Replace deterministic add-ons with a real provider	12
12	Conclusion	12

1 Objectives

The goal was to move the ResonantOS SDK from a verified contract to a browser-visible demonstration that *consumes* the platform’s real mechanisms rather than inventing parallel ones. The demonstration had to answer one question:

Can a third-party-style add-on be authored against the public `@resonantos/addon-sdk` contract, discovered and validated by the host, registered into the host-owned registry, granted a capability through explicit operator consent, exercised inside the real browser extension, and revoked — returning a real host-policy 403 — all through one generic contract, with no add-on-specific Core code and no second trust model?

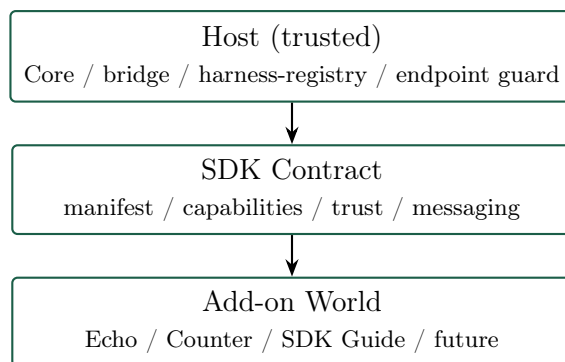
The acceptance criteria were strict:

- **Generic discovery** — the host scans a canonical location and picks up independently packaged add-ons without teaching Core their identifiers.
- **Manifest-driven contract** — identity, version, capabilities, and trust posture come from the add-on’s own `addon.json`, validated by `validateAddOnManifest`.
- **No self-grant** — requested capabilities are authored with `granted: false`; grants are minted by the host-owned registry only after `consent: true`.
- **Isolation** — the add-on UI runs in a sandboxed cross-origin iframe and cannot reach extension internals.

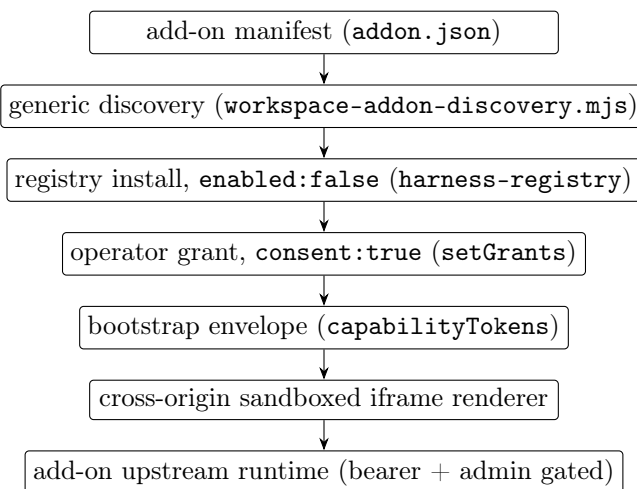
- **Real enforcement** — an authorized call returns 200, a wrong credential returns 401, and a revoked grant returns 403 from the upstream, not a renderer-side simulation.
- **Replacement proof** — at least two unrelated add-ons onboard through the same mechanism, and a third tutorial add-on teaches the SDK while itself running through that mechanism.

2 Architecture Overview

The SDK is a *plug/socket* contract layered over ResonantOS architecture, not a replacement for it. On one side is the trusted host (Core, the bridge, the host-owned registry, the endpoint guard); on the other is the replaceable add-on world; between them is the SDK boundary through which every interaction must travel.



The lifecycle a workspace add-on traverses is:



2.1 Authority

The single grant authority is the existing **harness-registry** (`createHarnessRegistry`), the same registry that already gates harness adapter routes. Workspace add-ons were extended onto it *without* inventing a second registry:

- `executeAddonsStatus` installs every discovered manifest with `enabled: false`.

- `POST /addons/workspace/grant` proxies `registry.setGrants(addonId, grants, { consent: true, expectedRevision })`; `consent: false` throws `permission-denied` at the registry boundary.
- `POST /addons/workspace/bootstrap` returns `capabilityTokens[cap].token` only for capabilities the host has granted.

2.2 Isolation and token delivery

Add-ons render inside a sandboxed iframe whose `src` is the add-on's own upstream origin (for example `http://127.0.0.1:47321`), using `sandbox="allow-scripts allow-same-origin"`. Because the iframe is same-origin with its own upstream and cross-origin from the extension, it can call its own API without CORS yet cannot reach the extension's `chrome.*` APIs, storage, or the bridge. No `Access-Control-Allow-Origin` header is emitted by any upstream.

Capability tokens are delivered from the trusted extension to the iframe via `postMessage` with `targetOrigin` pinned to the add-on's origin. The add-on validates both `event.source` and the message type before trusting it. The bridge token, the admin token, and any provider credential never reach the add-on.

3 The Reference Add-ons

Three add-ons were built, all registered through the same generic mechanism with zero Core changes:

Add-on	ID	Port	Purpose
Resonant Echo	<code>addon.resonant-echo</code>	47321	Deterministic echo; proves the bearer-gated round-trip
Resonant Counter	<code>addon.resonant-counter</code>	47322	Second add-on; proves the mechanism is generic
SDK Guide	<code>addon.sdk-guide</code>	47323	9-step tutorial; teaches the SDK while using it

3.1 Resonant Echo

A deterministic echo responder. Its mutating route `POST /api/echo/message` is bearer-gated; it returns the message verbatim as `Resonant Echo received: "<message>"`. It carries no model, provider, or network dependency.

3.2 Resonant Counter

A deterministic stateful counter exposing `POST /api/counter/{increment,decrement,reset}` and a public `GET /api/counter/value`. State lives only in the upstream's memory; the host never reads or writes it. Its existence is the *replacement proof*: a second, unrelated add-on onboards through the exact same mechanism with no additional Core code.

3.3 SDK Guide

A tutorial add-on that walks the developer through discovery, manifest validation, capability request, host consent, grant, sandboxed rendering, an authorized call, a denied call, and revocation. Each step is a *real* HTTP outcome from the operator-started guide upstream — step 7 reports 200, step 8 reports 401 (wrong bearer), and step 9 reports 403 (host-revoked). This is the strongest form of the replacement proof: an add-on that teaches the SDK while itself being an SDK add-on.

4 Capability and Security Model

The public capability vocabulary is `ADDON_CAPABILITIES` (14 entries): `filesystem`, `archive-read`, `archive-intake-write`, `chat-interface`, `memory-provider`, `providers`, `shell`, `network`, `ui-embedding`, `browser-control`, `agent-delegation`, `agent-runtime`, `notifications`, and `device-integration`. The three reference add-ons request only `network`, authored with `granted: false`.

Each grant is a `CapabilityGrant { capability, granted, scope, revocationBehavior }` with `CapabilityScope (none|self|workspace|shared|system|intake-only)` and `RevocationBehavior (hard-stop|degrade|hide-surface)`.

The status-code contract is explicit and exercised end-to-end:

Status	Meaning
200	operation succeeded with a valid bearer
401	caller presented a missing or wrong bearer (audience-bound credential boundary)
403	host revoked the grant; the bearer is valid but policy is closed (real host-policy result)
503	upstream started without an operator-pinned bearer; no mutating call is authorized

Two distinct operator-pinned tokens per add-on enforce the boundary:

- **Bearer token** — delivered to the iframe via the bootstrap envelope and carried as `Authorization: Bearer` on every mutating request. Held by the bridge; never in HTML or a URL.
- **Admin token** — gates `POST /admin/deny` on the upstream. Never delivered to the iframe; the bridge uses it out-of-band to flip the upstream's in-memory deny flag.

Token comparison in the upstreams is a constant-time XOR compare; wrong, case-shifted, and appended-byte tokens are rejected identically.

5 How It Was Built

The work was delivered in nine phases (P0–P9), each independently verified before the next was authorized.

Phase	Delivered
P0	baseline frozen on <code>upstream/dev @ 19665c06</code> ; full gate recorded
P1	architecture inventory; mapped 002 findings to current-dev disposition
P2	scaffold + validating Echo manifest
P3	Echo round-trip through the real extension
P4	Counter add-on + cross-add-on isolation
P5	sandboxed cross-origin renderer (<code>createWorkspaceAddonIframe</code>)
P6	host-owned grants; 401/403 lifecycle; grant-wipe regression fixed
P7	SDK Guide add-on (9-step tutorial with real policy denials)
P8	adversarial red-team (11-attack matrix) + route-audit gap closed
P9	live demo; README-only reproduction + walk-the-demo npm script

6 What Was Found

The independent review loop surfaced and corrected a series of real defects that the initial automated suite did not catch. These are recorded because they explain *why* the demonstration now works and why a browser-visible test is a necessary gate.

6.1 Resolved during the build

ID	Finding	Resolution
R1	Self-grant placeholder: bootstrap tokens were derived from author-controlled <code>grantPresets (granted: true)</code> .	Closed in P6: grants sourced from <code>harness-registry</code> , never <code>grantPresets</code> .
R2	CP4 false-green: the Counter live test reported 1/1 but failed on a stale <code>bridge-config.generated.js</code> .	Closed: stale-config cleanup added, matching Echo.
R3	CP3 deferred the real-extension test and substituted a network smoke test.	Closed: <code>echo-extension-live.test.mjs</code> loads the real unpacked extension.
R4	<code>apiBasePath</code> carried an origin rather than a path; a dead no-op loop remained.	Closed: field dropped, loop removed.
R5	Grant-wipe on re-poll: re-running <code>registry.install()</code> on every <code>/addons/status</code> poll reset grants.	Closed: guarded install + regression test.

6.2 Verification-quality corrections

Independent re-verification of the adversarial suite found several test titles that overstated what each test alone proved (no security hole, but honest labels matter). All were corrected and recorded as V1–V6:

- The “12 attacks” claim was actually 11 rows; the open-mirror/proxy concern was made an explicit negative assertion (V1).
- The use-after-revoke test drives only the admin channel; the both-channel witness is the real-extension test (V2).
- The forged-postMessage tests are static shape checks, not runtime forgery tests (V3).
- The “constant-time” test asserts rejection, not a timing measurement (V4).
- The port-collision test’s launcher-outcome value is now asserted, with the manifest-unchanged invariant stated honestly (V5).
- `runtime.command` is inert by construction (declarative discovery, no spawn path), now stated explicitly (V6).

6.3 Deferred items (recorded, not lost)

ID	Item
D1	Host-mediated enforcement: the iframe still calls its own upstream directly, so the host is not the only path a mutating request can take.
D2	Operator-pinned bearer tokens are static, not host-minted, expiring, audience-bound credentials.
D3	Revocation is an out-of-band <code>/admin/deny</code> signal, not the host-mediated production shape.
D4	Real-extension tests are not in the default CI run.
D5	Pre-existing <code>openai-harness-adapter</code> timing flake, unrelated to the demo.
D6	<code>harness-registry.install()</code> is non-idempotent (re-install resets grants).

7 Adversarial Red-Team

The trust boundaries were red-teamed. Every attack fails safely; no attack is “passed” by weakening a control, and no mock bridge skips auth/CORS/origin checks. The matrix covers the 12 red-team concerns from the checklist, consolidated into 11 named attacks:

#	Attack	Defensive control
1	Secrets leak	bootstrap envelope carries only the host-minted bearer; no token in HTML/URL/env.
2	Cross-add-on credential reuse	per-add-on audience-bound bearer; 3-way isolation.
3	Use-after-revoke	two independent channels (registry + admin); bearer dead until both restored.
4	Self-grant / outside-capabilities / no consent	registry permission-denied + optimistic concurrency.
5	Forged <code>postMessage</code>	<code>event.source</code> and type checks; parent pins <code>targetOrigin</code> .
6	Arbitrary entripoint / non-loopback / non-local-service	discovery rejects; no spawn path.
7	Open mirror/proxy route	explicit negative assertion: no <code>/addons/workspace/proxy*</code> route.
8	Malformed credentials	strict constant-time compare; 503/401/200 distinction.
9	Crash + bridge restart	durable registry journal preserves grants.
10	Port collision	bridge never rewrites the manifest entripoint.
11	Hermes/OpenCode/Livinghost Archive displacement	most services remain wired and capability-gated.

8 Test Results

All suites are green on Node v26.0.0 (the repository requires $\geq 24.21.0$):

Suite	Result
<code>npx vitest run</code>	721/721
<code>npx vitest run -config examples/sdk-demo/vitest.config.ts</code>	58/58
<code>npm run test:browser-host</code>	13/13
<code>sd003-p8-adversarial.test.mjs</code>	10/10
<code>bridge-route-capability-audit.test.mjs</code>	5/5
<code>harness-registry-workspace-addon.test.mjs</code>	8/8
<code>addons-status-grant-regression.test.mjs</code>	2/2
<code>npm run test:sdk-demo:extension</code> (four live tests)	4/4
Total	821

The four real-extension tests are the strongest evidence: each boots its own upstream and bridge, loads the actual unpacked extension in Chrome, and drives the lifecycle against the real host boundary — no `-disable-extensions`, no `Page.addScriptToEvaluateOnNewDocument`, no mock bridge that skips auth.

9 User Guide

This section describes how to reproduce the demonstration from a clean checkout.

9.1 Prerequisites

- Node.js >=24.21.0 (see .nvmrc; developed on v26.0.0).
- A Chromium-family browser (Chrome, Edge, Brave, Arc).

9.2 Checkout and install

```
git clone https://github.com/vonstegen/2.0.0-alpha.git
cd 2.0.0-alpha
git checkout r-and-d/sdk-demo-003-current-dev
npm ci
```

9.3 Start the three upstreams

Open three terminals; the bridge never spawns them.

```
node examples/sdk-demo/echo/server.mjs \
  --echo-bearer-token=demo-echo-bearer \
  --echo-admin-token=demo-echo-admin

node examples/sdk-demo/counter/server.mjs \
  --counter-bearer-token=demo-counter-bearer \
  --counter-admin-token=demo-counter-admin

node examples/sdk-demo/sdk-guide/server.mjs \
  --sdk-guide-bearer-token=demo-guide-bearer \
  --sdk-guide-admin-token=demo-guide-admin
```

9.4 Start the bridge

```
node browser-first/host/run-bridge-minimal.mjs \
  --bridge-port=47300 \
  --bridge-token=demo-bridge-token \
  --capability-bootstrap-token=demo-bootstrap-token \
  --addon-runtime-read-token=demo-addon-runtime-read \
  --addon-runtime-control-token=demo-addon-runtime-control \
  --echo-bearer-token=demo-echo-bearer \
  --echo-admin-token=demo-echo-admin \
  --counter-bearer-token=demo-counter-bearer \
  --counter-admin-token=demo-counter-admin \
  --sdk-guide-bearer-token=demo-guide-bearer \
  --sdk-guide-admin-token=demo-guide-admin
```

(The full capability-token flag set is the `BRIDGE_CAPABILITY_TOKEN_SPECS` table in `browser-first/host/bridge-capability-tokens.mjs`; the demo-relevant subset above is sufficient.)

9.5 Load the extension

1. In Chrome, open `chrome://extensions`.
2. Enable **Developer mode**.
3. Choose **Load unpacked** and select `browser-first/resonantos-side-panel-extension`.
4. Open the ResonantOS side panel and navigate to **Add-ons**.

9.6 Walk the lifecycle

1. **Open Resonant Echo**, type Hello Manolo, and press **SEND**; the response Resonant Echo received: "Hello Manolo" renders.
2. **Grant** an add-on; the registry records the grant and the add-on can call its upstream (200).
3. Present a **wrong bearer**; the upstream returns 401.
4. **Revoke**; the upstream returns 403 (real host policy).
5. **Admin-revoke + re-grant**; the upstream returns 200 again.
6. Switch to **Hermes**, **OpenCode**, and **Living Archive**; each remains present and capability-gated.

9.7 Run the test suite

```
npm run test:sdk-demo:vitest          # 58/58 deterministic
npm run test:sdk-demo:extension        # 4/4 real-browser
npx vitest run                        # 721/721
npm run test:browser-host              # 13/13
node --test --test-concurrency=1 browser-first/test/sd003-p8-adversarial.test.mjs
```

10 Demo-vs-Production Honesty

The demonstration is deliberately honest about what it does and does not prove. The enforcement point in the demo is the *add-on's own upstream*: a bearer-gated mutating route plus an admin-gated in-memory deny flag, with the bridge calling `/admin/deny` out-of-band. This is the cleanest mechanism that fits an operator-started local-service add-on without inventing a second host transport.

It proves: (a) the host-owned registry is the right authority; (b) consent-gated grant and revoke round-trip through the registry; (c) the per-add-on audience boundary holds; and (d) the upstream distinguishes 401 (wrong bearer) from 403 (host-revoked).

It does *not* prove: (e) the host is the *only* path a mutating request can take — the iframe still has direct loopback access to its own upstream. Closing (e) is the central item in the way forward.

11 The Way Forward Toward a Running SDK Platform

The demonstration is a verified boundary, not a finished product. The following sequence turns it into a production running SDK platform.

11.1 Close the enforcement gap (D1 + D3)

Route the iframe’s mutating requests through a bridge-owned proxy — `POST /addons/workspace/proxy/{addonId}/api/...` — that is the *only* listener on the upstream’s loopback port, validates the grant before forwarding, and injects a host-side token. The iframe then has no direct loopback path, so the host becomes the sole enforcement point, and revocation collapses into “stop forwarding” at the bridge boundary rather than an add-on-side flag.

11.2 Token hygiene (D2)

Replace the static operator-pinned bearer with host-minted, expiring, audience-bound per-grant tokens, rotated on revoke. The harness-boundary machinery already mints signed session references; workspace add-ons should consume the same model rather than a shared static value.

11.3 Registry idempotency (D6)

Decide and implement `harness-registry.install()` semantics so an identical re-install either no-ops or rejects, removing the latent footgun that re-install resets grants.

11.4 CI wiring (D4)

Promote the four real-extension tests into the default gate with a skip-when-no-browser policy, so the browser-visible lifecycle cannot silently drop out of CI.

11.5 Flake hygiene (D5)

Re-verify `openai-harness-adapter.test.mjs` on the pinned Node 24.21.0 and fix or exclude it independently of the demo.

11.6 Merge path

After the P10 integration review (diff audit against `upstream/dev`, full 002-findings disposition, and PR notes), fast-forward the branch into the canonical fork, then reconcile with `dev` — no force-push, no push to the ResonantOS org without review.

11.7 Publish the SDK contract

Publish `@resonantos/addon-sdk` to npm so add-on authors consume a versioned, stable contract rather than a repository path. This is the author-facing boundary that makes the demonstration a platform.

11.8 Replace deterministic add-ons with a real provider

Swap the deterministic Echo/Counter responders for a real provider-backed add-on (the existing `openai-compatible-harness` reference manifest is the pattern), reusing the same discovery/registry/bridge/extension socket. This proves the boundary carries real work, not just a canned round-trip.

12 Conclusion

SDK-DEMO-003 proves, in the real browser extension, that a third-party-style add-on can be discovered, validated, registered, granted, enforced, and revoked through a single generic contract with no add-on-specific Core code and no parallel trust model. Three add-ons — Resonant Echo, Resonant Counter, and the SDK Guide — onboard through the same mechanism; grants are host-owned and consent-gated; a wrong credential returns 401 and a revoked grant returns a real 403; the iframe is isolated; the adversarial matrix fails safe; and the full 821-test gate is green. The remaining work — host-mediated enforcement, token rotation, CI wiring, and npm publication — is enumerated, and the demonstration is positioned as the verified boundary on which that production platform is built.